

PROJECT CONTEXT

We are implementing a cybersecurity research PoC inspired by the paper:

"A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control"

However, we are NOT implementing the entire paper.

After extensive analysis, the scope was intentionally reduced to focus on a specific security problem that is both meaningful and implementable within an internship timeline.

FINAL PROBLEM STATEMENT

Traditional delegation mechanisms (OAuth-style token inheritance and similar approaches) are insufficient for autonomous multi-agent systems because:

- Agents are ephemeral and dynamically spawned.
- Delegation chains become difficult to control.
- Subagents may receive excessive privileges.
- Credential ownership is weak.
- Stolen credentials can be replayed.
- Authorization requirements change depending on context.

The PoC aims to demonstrate that:

"Policy-governed constrained delegation combined with JIT Verifiable Credentials and Proof of Possession provides safer delegation for ephemeral AI agents."

WHAT IS NOT IN SCOPE

The following are intentionally excluded:

- Full runtime trust analytics.
- Continuous behavioral monitoring.
- Autonomous risk scoring.
- Large-scale multi-agent orchestration.
- Distributed telemetry systems.
- Complete decentralized infrastructure.
- Blockchain implementation.
- Production-grade scalability.

The goal is a focused Proof of Concept, not a production platform.

ARCHITECTURE TO IMPLEMENT

The architecture contains four layers.

1. Agent Layer
2. Policy Layer
3. Credential Layer
4. Verification Layer

AGENT LAYER

Parent Agent:

- Owns DID.
- Owns key pair.
- Holds active VC.
- Possesses permissions delegated by issuer.
- Requests delegation for subagents.

Subagent:

- Has its own DID.
- Has its own key pair.
- Initially has no permissions.
- Receives JIT VC if delegation is approved.

POLICY LAYER

Consists of:

PIP PDP PAP

PAP (Policy Administration Point)

Stores predefined organizational policies.

Examples:

- Maximum delegation depth.
- Delegable permissions.
- Emergency access policies.
- Expiry limits.
- Resource constraints.
- Purpose restrictions.

Policies are static and predefined.

PAP does NOT generate policies dynamically.

PIP (Policy Information Point)

In a real system, PIP would gather runtime context.

However, because this PoC does not include a full running agent ecosystem, context will be simulated.

We explicitly decided to use predefined context scenarios.

Example context fields:

- emergency_active
- delegation_depth
- resource_sensitivity
- parent_vc_valid
- issuer_trusted
- hospital_status
- requested_permissions
- requested_expiry
- request_time
- subagent_count

Approximately 5–10 predefined scenarios may be created.

PIP simply returns context facts.

It does not make decisions.

PDP (Policy Decision Point)

Evaluates whether delegation should be allowed.

PDP checks:

1. Context prerequisites satisfied?
2. Requested permissions allowed by PAP?
3. Requested permissions subset of parent permissions?
4. Delegation depth allowed?
5. Expiry constraints valid?
6. Purpose constraints valid?

PDP outputs:

ALLOW or DENY

IMPORTANT DESIGN DECISION

Constrained delegation alone is not sufficient.

Even if:

$\text{delegated_permissions} \subseteq \text{parent_permissions}$

the delegation may still be denied because contextual prerequisites are not satisfied.

Example:

Parent owns MRI access.

Policy says MRI delegation only during emergency.

Emergency flag = false.

Result:

Delegation denied.

CREDENTIAL LAYER

Issuer Authority:

- Trusted issuer.
 - Receives PDP approval.
 - Creates JIT VC.
 - Signs VC.
-

JIT VC STRUCTURE

Fields include:

- vc_id
- issuer
- subject
- parent_agent
- permissions
- resource_constraints
- purpose
- issued_at
- expiry
- delegation_depth
- transferable = false
- issuer_signature

Purpose of VC:

- Short-lived.

- Scoped.
 - Delegation-aware.
 - Traceable.
-

REVOCATION REGISTRY

For MVP:

Centralized.

Stores:

- revoked VCs
- revoked DIDs
- compromised entities

Verifier checks revocation before granting access.

VERIFICATION LAYER

Contains:

- Subagent
 - Proof of Possession
 - Resource Server
-

PROOF OF POSSESSION

PoP was introduced after identifying a security gap.

Possession of a VC should not automatically grant trust.

Problem:

Attacker steals a valid VC.

Without ownership verification:

Attacker can replay credential.

Solution:

Bind credential usage to DID ownership.

Conceptual flow:

1. Verifier sends challenge.
2. Agent signs challenge using DID private key.
3. Verifier validates signature using DID public key.
4. Ownership proven.

Important:

Detailed PoP implementation has intentionally been postponed.

The current implementation order focuses first on:

- DID generation
- VC issuance
- Delegation validation
- JIT VC workflow

PoP implementation will be added later.

IMPLEMENTATION ORDER

Phase 1

- DID generation
- Key pair generation

Phase 2

- VC schema
- VC signing
- VC verification

Phase 3

- Delegation request model
- PAP/PIP/PDP implementation
- Constrained delegation checks

Phase 4

- JIT VC issuance flow

Phase 5

- Success and failure scenarios

Phase 6

- Proof of Possession implementation
-

EXPECTED DEMONSTRATIONS

Success Case:

- Delegation approved.
- JIT VC issued.
- Access granted.

Failure Case 1:

- Requested permissions exceed parent permissions.

Failure Case 2:

- Context prerequisites not satisfied.

Failure Case 3:

- Delegation depth exceeded.

Failure Case 4:

- VC revoked.

Failure Case 5:

- Proof of Possession fails.

PRIMARY SECURITY CONTRIBUTION

The PoC demonstrates:

Policy-governed constrained delegation using JIT Verifiable Credentials, with future integration of Proof of Possession, for safer authorization of ephemeral AI agents.